Spark — Universal Intelligence System

README.md

````
# Spark — Universal Intelligence System

Spark is a standalone, product-agnostic intelligence platform. It is **not**
a chatbot, not a UI, and not tied to any single product. Products (Bud, a
future Realm Assistant, a future Orbit Assistant, etc.) consume Spark's
intelligence through one stable SDK boundary while keeping their own
personality and purpose.

```ts
import { createSpark } from "@/lib/spark";

const spark = createSpark();
await spark.initialize();

spark.identity.createContext({ userId: "u1", sessionId: "s1", product: "bud" });
const results = await spark.search.search("refund policy");
const plan = spark.planning.createPlan("Answer the user's question", ["Search kno
````

Consumers should only ever `import { ... } from "@/lib/spark"`. Nothing under `spark/core/*`, `spark/memory/*`, `spark/intelligence/*`, etc. is a supported import path — those are internal and may change.

Status

This is Phase 1: architecture, interfaces, and local (offline, no API key, no network call) implementations for every module. Real hosted model providers (OpenAI, Anthropic, Gemini) are wired as placeholder adapters behind the `AIProvider` interface but are not yet implemented — Spark runs entirely on the built-in `MockProvider` until you register a real one.

Verified in this repo:

- `npm run typecheck` **— passes with strict TypeScript**
- `npm run smoke-test` **— exercises all 18 services end to end**
````

Architecture

```
spark/
  container.ts        DI container (per-instance, lazy singletons)
  events.ts           Internal event bus (memory.updated, search.completed, ...)
  middleware.ts       Middleware pipeline (logging, auth, cache, rate-limit)
  plugins.ts          Plugin registration (extend Spark without modifying core)
  manifest.ts         Static metadata (version, capabilities)
  types.ts            Health/metrics types
  index.ts            Public SDK — the ONLY supported import path

  core/
    identity/         User, session, product, locale, timezone, device context
    reasoning/        Analysis and decisions, with explainable steps
    planning/         Goals, task sequencing, dependency-aware plans

  memory/             Short-term, long-term, semantic, episodic, workspace
  context/            Session/environment awareness (separate from identity)
  knowledge/          Document store + naive relevance query (swap for a
                      real knowledge graph / embeddings store later)

  intelligence/
    search/           Depends only on Knowledge (Search -> Knowledge)
    recommendations/  Tag-overlap scoring
    organization/     Grouping / sorting
    personalization/  Per-user preference store
    writing/          Delegates to AIProvider
    translation/      Delegates to AIProvider
    summaries/        Deterministic extractive summarizer

  trust/
    privacy/          PII detection + redaction (regex-based)
    security/         Heuristic injection/scam-pattern scanner

  automation/         Named task registration + execution + history
  learning/           Feedback capture + aggregate scoring

  providers/
    types.ts          AIProvider interface — the ONLY thing services call
    registry.ts       Registration, default selection, availability fal
    mock.ts           Default provider — deterministic, offline, alwa
    openai.ts / anthropic.ts /   Placeholder adapters. No SDK dependency, no netw
    gemini.ts / local.ts         calls. Implement complete() when ready to go l
```

Dependency direction

Services depend downward and never sideways in a way that would create cycles:

```
Search  ->  Knowledge
Writing / Translation / Reasoning  ->  ProviderRegistry (AIProvider interface)
```

Spark never imports any product (Bud, UNIBUD, My Realm, Orbit). Products only ever import `@/lib/spark`.

Lifecycle

```
await spark.initialize(); // resolves every registered service once, eagerly
await spark.reset();      // clears the container and re-registers core services
await spark.shutdown();   // clears event listeners and middleware
```

Introspection

```
spark.version();       // "0.1.0"
spark.modules();       // registered service tokens
spark.capabilities();  // ["identity", "reasoning", ..., "learning"]
spark.manifest();      // name, version, build, capabilities, modules, providers
spark.health();        // status, loaded modules, provider availability, diagnost
spark.metrics();       // memory size, events emitted (extend as real counters ar
```

Adding a real provider

```
import { createSpark, OpenAIProvider } from "@/lib/spark";

const spark = createSpark();
spark.registerProvider(new OpenAIProvider(process.env.OPENAI_API_KEY), /* makeDef
```

Until `OpenAIProvider.complete()` is implemented, `ProviderRegistry.resolve()` will keep silently falling back to `MockProvider` when the registered provider reports `isAvailable() === false` — Spark never throws just because a real provider isn't configured yet.

Extending Spark

- **Middleware —** `spark.use(loggingMiddleware)` or write your own `(ctx, next) => Promise<T>` function for auth, caching, rate limiting.

- **Plugins —** `spark.registerPlugin({ name, install(ctx) { ... } })` **to add services, providers, or middleware without touching Spark's core.**
- **Events —** `spark.events.on("search.completed", handler)` **. Note: no service currently emits domain events yet; wire** `events.emit(...)` **calls into individual services as needed.**

Explicitly out of scope for this phase

- **No hosted model calls (all providers besides Mock throw until implemented)**
- **No persistence (everything is in-memory and resets on process restart)**
- **No product integration (Bud/UNIBUD/My Realm/Orbit are not referenced anywhere)**
- **No UI of any kind**

````
## `package.json`

```json
{
  "name": "@ecosystem/spark",
  "version": "0.1.0",
  "description": "Spark — the Universal Intelligence System. Product-agnostic, pr
  "main": "src/lib/spark/index.ts",
  "types": "src/lib/spark/index.ts",
  "scripts": {
    "typecheck": "tsc --noEmit",
    "build": "tsc",
    "smoke-test": "ts-node scripts/smoke-test.ts"
  },
  "devDependencies": {
    "typescript": "^5.5.4",
    "ts-node": "^10.9.2",
    "@types/node": "^20.14.0"
  }
}
````

tsconfig.json

```
{
  "compilerOptions": {
```
```

```
    "target": "ES2020",
    "module": "CommonJS",
    "moduleResolution": "node",
    "ignoreDeprecations": "6.0",
    "lib": ["ES2020", "DOM"],
    "declaration": true,
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "baseUrl": ".",
    "paths": {
      "@/lib/spark": ["src/lib/spark/index.ts"],
      "@/lib/spark/*": ["src/lib/spark/*"]
    }
  },
  "include": ["src/**/*.ts", "scripts/**/*.ts"]
}
```

scripts/smoke-test.ts

```
import { createSpark, loggingMiddleware } from "../src/lib/spark";

async function main() {
  const spark = createSpark();
  spark.use(loggingMiddleware);
  await spark.initialize();

  console.log("=== manifest ===");
  console.log(JSON.stringify(spark.manifest(), null, 2));

  console.log("\n=== capabilities ===");
  console.log(spark.capabilities());

  // Identity
  const identity = spark.identity.createContext({
    userId: "user_1",
    sessionId: "session_1",
    product: "smoke-test",
  });
  console.log("\n=== identity ===", identity);
```

```
// Memory
spark.memory.remember({
  kind: "short_term",
  content: "The user asked about Spark's architecture.",
  sessionId: "session_1",
  userId: "user_1",
});
console.log("\n=== memory recall ===", spark.memory.recall({ sessionId: "sessio

// Context
spark.context.setAttribute("session_1", "currentScreen", "dashboard");
console.log("\n=== context snapshot ===", spark.context.getSnapshot("session_1"

// Knowledge + Search
spark.knowledge.addDocument({
  title: "Spark Overview",
  content: "Spark is the Universal Intelligence System that powers products acr
  tags: ["spark", "architecture"],
});
const searchResults = await spark.search.search("universal intelligence");
console.log("\n=== search ===", searchResults);

// Reasoning
const reasoning = await spark.reasoning.analyze({
  question: "Should Bud call Spark or an AI provider directly?",
  facts: ["Bud must not contain intelligence or business logic."],
});
console.log("\n=== reasoning ===", reasoning);

// Planning
const plan = spark.planning.createPlan("Ship Spark v1", [
  "Define public API",
  "Implement local services",
  "Run production review",
]);
console.log("\n=== planning ===", plan, spark.planning.nextTask(plan.id));

// Recommendations
console.log(
  "\n=== recommendations ===",
  spark.recommendations.recommend({
    candidateItems: [
      { id: "a", tags: ["spark"] },
      { id: "b", tags: ["unrelated"] },
    ],
    basedOnTags: ["spark"],
  })
```

```
);

// Organization
console.log(
  "\n=== organization ===",
  spark.organization.groupByTag([
    { id: "1", label: "doc a", tags: ["spark"] },
    { id: "2", label: "doc b", tags: ["bud"] },
  ])
);

// Personalization
spark.personalization.setPreference("user_1", "theme", "dark");
console.log("\n=== personalization ===", spark.personalization.getAllPreference

// Writing / Translation / Summaries (mock provider)
console.log("\n=== writing ===", await spark.writing.draft({ prompt: "Say hello
console.log(
  "\n=== translation ===",
  await spark.translation.translate({ text: "Hello", targetLocale: "es-ES" })
);
console.log(
  "\n=== summaries ===",
  spark.summaries.summarize({
    text: "Spark is the Universal Intelligence System. It powers Bud, Realm Ass
    maxSentences: 1,
  })
);

// Privacy / Security
console.log(
  "\n=== privacy scan ===",
  spark.privacy.scanForPii("Contact me at test@example.com")
);
console.log(
  "\n=== security check ===",
  spark.security.checkInput("Please verify your password immediately")
);

// Automation
spark.automation.registerTask("ping", async () => "pong");
console.log("\n=== automation ===", await spark.automation.run("ping"));

// Learning
spark.learning.recordFeedback({ subject: "writing", rating: "positive" });
console.log("\n=== learning score ===", spark.learning.score("writing"));
```

```
  // Health
  console.log("\n=== health ===", spark.health());

  await spark.shutdown();
  console.log("\nSmoke test completed successfully.");
}

main().catch((err) => {
  console.error("Smoke test failed:", err);
  throw err;
});
```

scripts/container-verify.ts

```
import { createSpark } from "../src/lib/spark";
import { Container } from "../src/lib/spark/container";
import { ServiceNotRegisteredError } from "../src/lib/spark/errors";

async function main() {
  const spark = createSpark();
  console.log("modules():", spark.modules());
  console.log("manifest().registeredModules:", spark.manifest().registeredModules

  // registerValue + missing-token error behavior, isolated from Spark's own cont
  const c = new Container();
  const sentinel = { ok: true };
  const token = Symbol("test.value");
  c.registerValue(token, sentinel);
  const resolved = c.resolve<typeof sentinel>(token);
  if (resolved !== sentinel) throw new Error("registerValue did not preserve iden
  console.log("registerValue OK, resolved instance === original:", resolved === s

  try {
    c.resolve(Symbol("never.registered"));
    throw new Error("expected ServiceNotRegisteredError to be thrown");
  } catch (err) {
    if (!(err instanceof ServiceNotRegisteredError)) {
      throw new Error(`expected ServiceNotRegisteredError, got ${err}`);
    }
    console.log("ServiceNotRegisteredError thrown correctly:", err.message);
  }

  // Per-instance isolation: two Spark instances never share a container
  const sparkA = createSpark();
```

```
  const sparkB = createSpark();
  sparkA.memory.remember({ kind: "short_term", content: "only in A" });
  const crossLeak = sparkB.memory.recall({ text: "only in A" });
  if (crossLeak.length !== 0) throw new Error("Spark instances are leaking state!
  console.log("Per-instance isolation OK: sparkB sees", crossLeak.length, "matchi

  console.log("\nContainer verification completed successfully.");
}

main().catch((err) => {
  console.error("Container verification failed:", err);
  throw err;
});
```

src/lib/spark/index.ts

```
/**
 * Public Spark SDK.
 *
 * Consumers should only ever import from here:
 *
 *   import { createSpark } from "@/lib/spark";
 *
 * Never import from spark/core/*, spark/memory/*, etc. directly —
 * those paths are internal implementation detail and may change.
 */

import { Container } from "./container";
import { TOKENS, tokenLabel } from "./tokens";
import { EventBus } from "./events";
import { MiddlewarePipeline, type Middleware } from "./middleware";
import { PluginManager, type SparkPlugin } from "./plugins";
import { ProviderRegistry } from "./providers/registry";
import type { AIProvider } from "./providers/types";
import {
  SPARK_CAPABILITIES,
  SPARK_BUILD,
  SPARK_VERSION,
  type SparkManifest,
} from "./manifest";
import type { SparkHealthReport, SparkMetrics } from "./types";

import { LocalIdentityService } from "./core/identity/local";
import { LocalReasoningService } from "./core/reasoning/local";
```

```
import { LocalPlanningService } from "./core/planning/local";
import { InMemoryMemoryService } from "./memory/local";
import { LocalContextService } from "./context/local";
import { LocalKnowledgeService } from "./knowledge/local";
import { LocalSearchService } from "./intelligence/search/local";
import { LocalRecommendationsService } from "./intelligence/recommendations/local
import { LocalOrganizationService } from "./intelligence/organization/local";
import { LocalPersonalizationService } from "./intelligence/personalization/local
import { LocalWritingService } from "./intelligence/writing/local";
import { LocalTranslationService } from "./intelligence/translation/local";
import { LocalSummariesService } from "./intelligence/summaries/local";
import { LocalPrivacyService } from "./trust/privacy/local";
import { LocalSecurityService } from "./trust/security/local";
import { LocalAutomationService } from "./automation/local";
import { LocalLearningService } from "./learning/local";

import type { IdentityService } from "./core/identity/interface";
import type { ReasoningService } from "./core/reasoning/interface";
import type { PlanningService } from "./core/planning/interface";
import type { MemoryService } from "./memory/interface";
import type { ContextService } from "./context/interface";
import type { KnowledgeService } from "./knowledge/interface";
import type { SearchService } from "./intelligence/search/interface";
import type { RecommendationsService } from "./intelligence/recommendations/inter
import type { OrganizationService } from "./intelligence/organization/interface";
import type { PersonalizationService } from "./intelligence/personalization/inter
import type { WritingService } from "./intelligence/writing/interface";
import type { TranslationService } from "./intelligence/translation/interface";
import type { SummariesService } from "./intelligence/summaries/interface";
import type { PrivacyService } from "./trust/privacy/interface";
import type { SecurityService } from "./trust/security/interface";
import type { AutomationService } from "./automation/interface";
import type { LearningService } from "./learning/interface";

export class Spark {
  readonly events = new EventBus();
  readonly middleware = new MiddlewarePipeline();

  private readonly container = new Container();
  private readonly providerRegistry = new ProviderRegistry();
  private readonly plugins = new PluginManager();
  private readonly startedAt = Date.now();
  private initialized = false;
  private readonly warnings: string[] = [];

  constructor() {
    this.registerCoreServices();
```

```
}

// ---------------------------------------------------------------------
// Lifecycle
// ---------------------------------------------------------------------

async initialize(): Promise<void> {
  if (this.initialized) return;
  // Eagerly resolve every registered service once so configuration
  // errors surface immediately rather than on first use.
  for (const token of this.container.registeredTokens()) {
    this.container.resolve(token);
  }
  this.initialized = true;
  this.events.emit("spark.initialized", { at: new Date().toISOString() });
}

async shutdown(): Promise<void> {
  this.events.emit("spark.shutdown", { at: new Date().toISOString() });
  this.events.clear();
  this.middleware.clear();
  this.initialized = false;
}

async reset(): Promise<void> {
  this.container.reset();
  this.registerCoreServices();
  this.initialized = false;
}

// ---------------------------------------------------------------------
// Providers & plugins
// ---------------------------------------------------------------------

registerProvider(provider: AIProvider, makeDefault = false): void {
  this.providerRegistry.register(provider, makeDefault);
}

registerPlugin(plugin: SparkPlugin): void {
  this.plugins.register(plugin, {
    container: this.container,
    providers: this.providerRegistry,
    middleware: this.middleware,
    events: this.events,
  });
}
```

```
use(middleware: Middleware): void {
  this.middleware.use(middleware);
}

// ---------------------------------------------------------------------
// Introspection
// ---------------------------------------------------------------------

version(): string {
  return SPARK_VERSION;
}

modules(): string[] {
  return this.container.registeredTokens().map(tokenLabel);
}

capabilities(): readonly string[] {
  return SPARK_CAPABILITIES;
}

manifest(): SparkManifest {
  return {
    name: "Spark",
    version: SPARK_VERSION,
    build: SPARK_BUILD,
    capabilities: SPARK_CAPABILITIES,
    registeredModules: this.container.registeredTokens().map(tokenLabel),
    providers: this.providerRegistry.list(),
  };
}

health(): SparkHealthReport {
  const diagnostics: string[] = [];
  const providers = this.providerRegistry.list();

  if (!providers.some((p) => p.available)) {
    this.warnings.push(
      "No AI provider is currently available besides the mock provider."
    );
  }

  diagnostics.push(
    `${this.container.resolvedTokens().length}/${
      this.container.registeredTokens().length
    } services resolved.`
  );
```

```
    const status: SparkHealthReport["status"] = this.initialized
      ? "healthy"
      : "degraded";

    return {
      status,
      initialized: this.initialized,
      loadedModules: this.container.resolvedTokens().map(tokenLabel),
      registeredProviders: providers,
      diagnostics,
      warnings: [...new Set(this.warnings)],
      uptimeMs: Date.now() - this.startedAt,
      checkedAt: new Date().toISOString(),
    };
  }

  metrics(): SparkMetrics {
    const events = this.events.recentEvents(500);
    return {
      requestCounts: {},
      executionTimesMs: {},
      cacheHits: 0,
      cacheMisses: 0,
      memorySize: this.memory.size(),
      eventsEmitted: events.length,
    };
  }

  // ---------------------------------------------------------------------
  // Service accessors — this is the actual public surface products use
  // ---------------------------------------------------------------------

  get identity(): IdentityService {
    return this.container.resolve(TOKENS.Identity);
  }

  get reasoning(): ReasoningService {
    return this.container.resolve(TOKENS.Reasoning);
  }

  get planning(): PlanningService {
    return this.container.resolve(TOKENS.Planning);
  }

  get memory(): MemoryService {
    return this.container.resolve(TOKENS.Memory);
  }
```

```
get context(): ContextService {
  return this.container.resolve(TOKENS.Context);
}

get knowledge(): KnowledgeService {
  return this.container.resolve(TOKENS.Knowledge);
}

get search(): SearchService {
  return this.container.resolve(TOKENS.Search);
}

get recommendations(): RecommendationsService {
  return this.container.resolve(TOKENS.Recommendations);
}

get organization(): OrganizationService {
  return this.container.resolve(TOKENS.Organization);
}

get personalization(): PersonalizationService {
  return this.container.resolve(TOKENS.Personalization);
}

get writing(): WritingService {
  return this.container.resolve(TOKENS.Writing);
}

get translation(): TranslationService {
  return this.container.resolve(TOKENS.Translation);
}

get summaries(): SummariesService {
  return this.container.resolve(TOKENS.Summaries);
}

get privacy(): PrivacyService {
  return this.container.resolve(TOKENS.Privacy);
}

get security(): SecurityService {
  return this.container.resolve(TOKENS.Security);
}

get automation(): AutomationService {
  return this.container.resolve(TOKENS.Automation);
```

```
}

get learning(): LearningService {
  return this.container.resolve(TOKENS.Learning);
}

// ---------------------------------------------------------------------
// Internal wiring
// ---------------------------------------------------------------------

private registerCoreServices(): void {
  this.container.registerValue(TOKENS.ProviderRegistry, this.providerRegistry);

  this.container.register(TOKENS.Identity, () => new LocalIdentityService());
  this.container.register(
    TOKENS.Reasoning,
    () => new LocalReasoningService(this.providerRegistry)
  );
  this.container.register(TOKENS.Planning, () => new LocalPlanningService());
  this.container.register(TOKENS.Memory, () => new InMemoryMemoryService());
  this.container.register(TOKENS.Context, () => new LocalContextService());
  this.container.register(TOKENS.Knowledge, () => new LocalKnowledgeService());

  // Search depends on Knowledge only — Search -> Knowledge, not Memory,
  // to keep the dependency graph shallow as Spark grows.
  this.container.register(
    TOKENS.Search,
    (c) => new LocalSearchService(c.resolve(TOKENS.Knowledge))
  );

  this.container.register(
    TOKENS.Recommendations,
    () => new LocalRecommendationsService()
  );
  this.container.register(
    TOKENS.Organization,
    () => new LocalOrganizationService()
  );
  this.container.register(
    TOKENS.Personalization,
    () => new LocalPersonalizationService()
  );
  this.container.register(
    TOKENS.Writing,
    () => new LocalWritingService(this.providerRegistry)
  );
  this.container.register(
```

```
      TOKENS.Translation,
      () => new LocalTranslationService(this.providerRegistry)
    );
    this.container.register(TOKENS.Summaries, () => new LocalSummariesService());
    this.container.register(TOKENS.Privacy, () => new LocalPrivacyService());
    this.container.register(TOKENS.Security, () => new LocalSecurityService());
    this.container.register(TOKENS.Automation, () => new LocalAutomationService()
    this.container.register(TOKENS.Learning, () => new LocalLearningService());
  }
}

/**
 * Factory for a new, fully independent Spark instance. Each call
 * produces its own container/providers/events/middleware — instances
 * never share state.
 */
export function createSpark(): Spark {
  return new Spark();
}

// Re-export types consumers are expected to use.
export type { AIProvider } from "./providers/types";
export type { SparkPlugin, SparkPluginContext } from "./plugins";
export type { Middleware } from "./middleware";
export type { SparkManifest } from "./manifest";
export type { SparkHealthReport, SparkMetrics } from "./types";
export { loggingMiddleware } from "./middleware";
export { MockProvider } from "./providers/mock";
export { OpenAIProvider } from "./providers/openai";
export { AnthropicProvider } from "./providers/anthropic";
export { GeminiProvider } from "./providers/gemini";
export { LocalModelProvider } from "./providers/local";
```

src/lib/spark/container.ts

```
/**
 * Minimal dependency-injection container.
 *
 * Services are registered as factories and resolved lazily on first
 * use, then cached as singletons for the lifetime of the Spark instance
 * that owns this container. Each `createSpark()` call constructs its
 * own Container, so multiple Spark instances never share state — there
 * is no module-level/global registry anywhere in this file.
 *
```

```
 * Canonical tokens live in ./tokens.ts — this file only consumes them.
 */

import type { Token } from "./tokens";
import { tokenLabel } from "./tokens";
import { ServiceNotRegisteredError } from "./errors";

type Factory<T> = (container: Container) => T;

export class Container {
  private factories = new Map<Token, Factory<unknown>>();
  private instances = new Map<Token, unknown>();

  /** Register a lazy factory. Only invoked on first resolve(). */
  register<T>(token: Token, factory: Factory<T>): void {
    this.factories.set(token, factory as Factory<unknown>);
    // Invalidate any cached instance so re-registration is respected.
    this.instances.delete(token);
  }

  /**
   * Register an already-constructed value directly (e.g. a shared
   * Logger or ProviderRegistry instance) without wrapping it in a
   * factory function. Still resolved lazily/cached like any other
   * token — resolve() doesn't distinguish how a token was registered.
   */
  registerValue<T>(token: Token, value: T): void {
    this.register(token, () => value);
  }

  has(token: Token): boolean {
    return this.factories.has(token);
  }

  resolve<T>(token: Token): T {
    if (this.instances.has(token)) {
      return this.instances.get(token) as T;
    }
    const factory = this.factories.get(token);
    if (!factory) {
      throw new ServiceNotRegisteredError(tokenLabel(token));
    }
    const instance = factory(this);
    this.instances.set(token, instance);
    return instance as T;
  }
```

```
  /** Tokens that currently have a registered factory. */
  registeredTokens(): Token[] {
    return Array.from(this.factories.keys());
  }

  /** Tokens that have actually been instantiated (lazy resolution check). */
  resolvedTokens(): Token[] {
    return Array.from(this.instances.keys());
  }

  reset(): void {
    this.instances.clear();
  }
}
```

src/lib/spark/tokens.ts

```
/**
 * The single canonical set of DI tokens used throughout Spark.
 *
 * Tokens are Symbols, not strings, so two independently-authored
 * modules can never collide on a token by accident even if they
 * happen to choose the same label. Every Symbol still carries its
 * original label as `description`, so diagnostics and logs stay
 * human-readable via `tokenLabel()` below.
 */

export const TOKENS = {
  Logger: Symbol("kernel.logger"),
  ProviderRegistry: Symbol("providers.registry"),

  Identity: Symbol("core.identity"),
  Memory: Symbol("core.memory"),
  Context: Symbol("core.context"),
  Knowledge: Symbol("core.knowledge"),
  Reasoning: Symbol("core.reasoning"),
  Planning: Symbol("core.planning"),

  Search: Symbol("intelligence.search"),
  Recommendations: Symbol("intelligence.recommendations"),
  Organization: Symbol("intelligence.organization"),
  Personalization: Symbol("intelligence.personalization"),
  Writing: Symbol("intelligence.writing"),
  Translation: Symbol("intelligence.translation"),
```

```
  Summaries: Symbol("intelligence.summaries"),

  Privacy: Symbol("trust.privacy"),
  Security: Symbol("trust.security"),

  Automation: Symbol("automation"),
  Learning: Symbol("learning"),
} as const;

export type Token = symbol;

/** Human-readable label for a token, for logs/diagnostics/health output. */
export function tokenLabel(token: Token): string {
  return token.description ?? token.toString();
}
```

src/lib/spark/errors.ts

```
/**
 * Spark's error hierarchy. Internal code should throw these instead of
 * generic `Error` so callers (and Kernel diagnostics) can distinguish
 * failure classes reliably.
 */

export class SparkError extends Error {
  constructor(message: string) {
    super(message);
    this.name = new.target.name;
    Object.setPrototypeOf(this, new.target.prototype);
  }
}

/** Thrown when Container.resolve() is called for an unregistered token. */
export class ServiceNotRegisteredError extends SparkError {
  constructor(public readonly tokenLabel: string) {
    super(`No service registered for token "${tokenLabel}".`);
  }
}

/** Thrown when a provider is looked up by name but was never registered. */
export class ProviderNotRegisteredError extends SparkError {
  constructor(public readonly providerName: string) {
    super(`Provider "${providerName}" is not registered.`);
  }
```

```
}

/** Thrown when a provider is resolved but cannot currently be used (e.g. no cred
export class ProviderUnavailableError extends SparkError {
  constructor(public readonly providerName: string, reason?: string) {
    super(
      `Provider "${providerName}" is not available${reason ? `: ${reason}` : "."}
    );
  }
}

/** Thrown by services when given malformed input. */
export class ValidationError extends SparkError {
  constructor(message: string) {
    super(message);
  }
}

/** Thrown when an operation requires Spark to be initialized first. */
export class NotInitializedError extends SparkError {
  constructor() {
    super("Spark has not been initialized. Call initialize() first.");
  }
}

/** Thrown when a plugin name is registered more than once. */
export class PluginAlreadyRegisteredError extends SparkError {
  constructor(public readonly pluginName: string) {
    super(`Plugin "${pluginName}" is already registered.`);
  }
}
```

src/lib/spark/events.ts

```
/**
 * Lightweight internal event bus.
 *
 * Modules publish events instead of calling each other directly, which
 * keeps them decoupled. External consumers (e.g. Oracle) can subscribe
 * without Spark needing to know they exist.
 *
 * Example event names: "memory.updated", "search.completed",
 * "recommendation.generated", "translation.finished".
 */
```

```
export type SparkEventName = string;

export type SparkEventHandler<TPayload = unknown> = (
  payload: TPayload
) => void;

export interface SparkEventLogEntry {
  name: SparkEventName;
  payload: unknown;
  timestamp: string;
}

export class EventBus {
  private handlers = new Map<SparkEventName, Set<SparkEventHandler>>();
  private log: SparkEventLogEntry[] = [];
  private readonly maxLog = 200;

  on<TPayload = unknown>(
    name: SparkEventName,
    handler: SparkEventHandler<TPayload>
  ): () => void {
    if (!this.handlers.has(name)) {
      this.handlers.set(name, new Set());
    }
    this.handlers.get(name)!.add(handler as SparkEventHandler);
    return () => this.off(name, handler);
  }

  off<TPayload = unknown>(
    name: SparkEventName,
    handler: SparkEventHandler<TPayload>
  ): void {
    this.handlers.get(name)?.delete(handler as SparkEventHandler);
  }

  emit<TPayload = unknown>(name: SparkEventName, payload?: TPayload): void {
    this.log.push({
      name,
      payload,
      timestamp: new Date().toISOString(),
    });
    if (this.log.length > this.maxLog) {
      this.log.shift();
    }
    this.handlers.get(name)?.forEach((handler) => {
      try {
```

```
        handler(payload);
      } catch (err) {
        // Event handlers must never break the emitting module.
        // eslint-disable-next-line no-console
        console.error(`[spark:events] handler for "${name}" threw`, err);
      }
    });
  }

  recentEvents(limit = 50): SparkEventLogEntry[] {
    return this.log.slice(-limit);
  }

  clear(): void {
    this.handlers.clear();
    this.log = [];
  }
}
```

src/lib/spark/middleware.ts

```
/**
 * Middleware pipeline.
 *
 * Middleware wraps every call made through Spark.call(), so cross-cutting
 * concerns (logging, auth, caching, rate limiting) can be added without
 * touching individual service implementations.
 */

export interface SparkCallContext {
  service: string;
  method: string;
  args: unknown[];
  startedAt: number;
  meta: Record<string, unknown>;
}

export type NextFn<TResult> = () => Promise<TResult>;

export type Middleware = <TResult>(
  ctx: SparkCallContext,
  next: NextFn<TResult>
) => Promise<TResult>;
```

```
export class MiddlewarePipeline {
  private middlewares: Middleware[] = [];

  use(middleware: Middleware): void {
    this.middlewares.push(middleware);
  }

  clear(): void {
    this.middlewares = [];
  }

  list(): number {
    return this.middlewares.length;
  }

  async run<TResult>(
    ctx: SparkCallContext,
    handler: () => Promise<TResult>
  ): Promise<TResult> {
    const chain = this.middlewares.reduceRight<NextFn<TResult>>(
      (next, mw) => () => mw(ctx, next),
      handler
    );
    return chain();
  }
}

/** Simple built-in logging middleware, opt-in via Spark.use(loggingMiddleware).
export const loggingMiddleware: Middleware = async (ctx, next) => {
  const result = await next();
  const durationMs = Date.now() - ctx.startedAt;
  // eslint-disable-next-line no-console
  console.log(
    `[spark] ${ctx.service}.${ctx.method} (${durationMs}ms)`
  );
  return result;
};
```

src/lib/spark/plugins.ts

```
import type { Container } from "./container";
import type { Token } from "./tokens";
import type { ProviderRegistry } from "./providers/registry";
import type { MiddlewarePipeline } from "./middleware";
```

```
import type { EventBus } from "./events";

export interface SparkPluginContext {
  container: Container;
  providers: ProviderRegistry;
  middleware: MiddlewarePipeline;
  events: EventBus;
}

export interface SparkPlugin {
  name: string;
  version?: string;
  /** Called once at registration time. Register services, providers, etc. here.
  install(ctx: SparkPluginContext): void;
}

export class PluginManager {
  private installed = new Map<string, SparkPlugin>();

  register(plugin: SparkPlugin, ctx: SparkPluginContext): void {
    if (this.installed.has(plugin.name)) {
      throw new Error(`Plugin "${plugin.name}" is already registered.`);
    }
    plugin.install(ctx);
    this.installed.set(plugin.name, plugin);
  }

  list(): Array<{ name: string; version?: string }> {
    return Array.from(this.installed.values()).map((p) => ({
      name: p.name,
      version: p.version,
    }));
  }
}

// Re-export for convenience so plugin authors have one import.
export type { Token };
```

src/lib/spark/manifest.ts

```
export const SPARK_VERSION = "0.1.0";
export const SPARK_BUILD = "dev";

export const SPARK_CAPABILITIES = [
```

```
  "identity",
  "reasoning",
  "planning",
  "memory",
  "context",
  "knowledge",
  "search",
  "recommendations",
  "organization",
  "personalization",
  "writing",
  "translation",
  "summaries",
  "privacy",
  "security",
  "automation",
  "learning",
] as const;

export type SparkCapability = (typeof SPARK_CAPABILITIES)[number];

export interface SparkManifest {
  name: string;
  version: string;
  build: string;
  capabilities: readonly SparkCapability[];
  registeredModules: string[];
  providers: Array<{ name: string; available: boolean; isDefault: boolean }>;
}
```

src/lib/spark/types.ts

```
export type HealthStatus = "healthy" | "degraded" | "unhealthy";

export interface SparkHealthReport {
  status: HealthStatus;
  initialized: boolean;
  loadedModules: string[];
  registeredProviders: Array<{
    name: string;
    available: boolean;
    isDefault: boolean;
  }>;
  diagnostics: string[];
```

```
  warnings: string[];
  uptimeMs: number;
  checkedAt: string;
}

export interface SparkMetrics {
  requestCounts: Record<string, number>;
  executionTimesMs: Record<string, number[]>;
  cacheHits: number;
  cacheMisses: number;
  memorySize: number;
  eventsEmitted: number;
}
```

src/lib/spark/core/identity/interface.ts

```
export interface DeviceContext {
  type?: "desktop" | "mobile" | "tablet" | "server" | "unknown";
  os?: string;
}

export interface IdentityContext {
  userId: string;
  sessionId: string;
  product: string;
  locale: string;
  timezone: string;
  device?: DeviceContext;
  createdAt: string;
}

export interface IdentityService {
  /** Create (or overwrite) the identity context for a session. */
  createContext(input: {
    userId: string;
    sessionId: string;
    product: string;
    locale?: string;
    timezone?: string;
    device?: DeviceContext;
  }): IdentityContext;

  getContext(sessionId: string): IdentityContext | undefined;
```

```
  updateContext(
    sessionId: string,
    patch: Partial<Omit<IdentityContext, "sessionId" | "createdAt">>
  ): IdentityContext | undefined;

  clearContext(sessionId: string): boolean;

  /** Number of active identity contexts currently tracked. */
  size(): number;
}
```

src/lib/spark/core/identity/local.ts

```
import type { IdentityContext, IdentityService, DeviceContext } from "./interface

/**
 * In-memory identity service. Provides user/session/product/locale/
 * timezone/device context for all other Spark services. No persistence
 * or network calls — state lives for the lifetime of the Spark instance.
 */
export class LocalIdentityService implements IdentityService {
  private contexts = new Map<string, IdentityContext>();

  createContext(input: {
    userId: string;
    sessionId: string;
    product: string;
    locale?: string;
    timezone?: string;
    device?: DeviceContext;
  }): IdentityContext {
    const context: IdentityContext = {
      userId: input.userId,
      sessionId: input.sessionId,
      product: input.product,
      locale: input.locale ?? "en-US",
      timezone: input.timezone ?? "UTC",
      device: input.device,
      createdAt: new Date().toISOString(),
    };
    this.contexts.set(input.sessionId, context);
    return context;
  }
```

```
  getContext(sessionId: string): IdentityContext | undefined {
    return this.contexts.get(sessionId);
  }

  updateContext(
    sessionId: string,
    patch: Partial<Omit<IdentityContext, "sessionId" | "createdAt">>
  ): IdentityContext | undefined {
    const existing = this.contexts.get(sessionId);
    if (!existing) return undefined;
    const updated = { ...existing, ...patch };
    this.contexts.set(sessionId, updated);
    return updated;
  }

  clearContext(sessionId: string): boolean {
    return this.contexts.delete(sessionId);
  }

  size(): number {
    return this.contexts.size;
  }
}
```

src/lib/spark/core/reasoning/interface.ts

```
export interface ReasoningInput {
  question: string;
  facts?: string[];
  context?: Record<string, unknown>;
}

export interface ReasoningStep {
  step: number;
  description: string;
}

export interface ReasoningResult {
  answer: string;
  confidence: number; // 0..1
  steps: ReasoningStep[];
  usedProvider: string;
}
```

```
export interface ReasoningService {
  /** Analyze input and produce a decision/answer with explainable steps. */
  analyze(input: ReasoningInput): Promise<ReasoningResult>;
}
```

src/lib/spark/core/reasoning/local.ts

```
import type {
  ReasoningInput,
  ReasoningResult,
  ReasoningService,
} from "./interface";
import type { ProviderRegistry } from "../../providers/registry";

/**
 * Deterministic local reasoning implementation. Produces a transparent,
 * step-by-step trace without requiring a hosted model. When a real
 * provider becomes available, this class can delegate to it through
 * ProviderRegistry without callers changing anything.
 */
export class LocalReasoningService implements ReasoningService {
  constructor(private readonly providers: ProviderRegistry) {}

  async analyze(input: ReasoningInput): Promise<ReasoningResult> {
    const provider = this.providers.resolve();
    const facts = input.facts ?? [];

    const steps = [
      { step: 1, description: `Parsed question: "${input.question}"` },
      {
        step: 2,
        description: facts.length
          ? `Considered ${facts.length} supplied fact(s).`
          : "No supporting facts were supplied.",
      },
      {
        step: 3,
        description: `Delegated synthesis to provider "${provider.name}".`,
      },
    ];

    const completion = await provider.complete({
      messages: [
        { role: "system", content: "You are Spark's reasoning module." },
```

```
        {
          role: "user",
          content: `Question: ${input.question}\nFacts: ${facts.join("; ")}`,
        },
      ],
    });

    return {
      answer: completion.text,
      confidence: facts.length > 0 ? 0.6 : 0.35,
      steps,
      usedProvider: provider.name,
    };
  }
}
```

src/lib/spark/core/planning/interface.ts

```
export interface PlanTask {
  id: string;
  title: string;
  done: boolean;
  order: number;
  dependsOn?: string[];
}

export interface Plan {
  id: string;
  goal: string;
  tasks: PlanTask[];
  createdAt: string;
}

export interface PlanningService {
  createPlan(goal: string, taskTitles: string[]): Plan;
  getPlan(planId: string): Plan | undefined;
  completeTask(planId: string, taskId: string): Plan | undefined;
  nextTask(planId: string): PlanTask | undefined;
}
```

src/lib/spark/core/planning/local.ts

```
import type { Plan, PlanTask, PlanningService } from "./interface";

/**
 * In-memory planning service. Handles goal decomposition into ordered,
 * dependency-aware tasks. No AI provider is required for the core
 * sequencing logic; reasoning-assisted decomposition can be layered on
 * top later via the ReasoningService.
 */
export class LocalPlanningService implements PlanningService {
  private plans = new Map<string, Plan>();
  private counter = 0;

  createPlan(goal: string, taskTitles: string[]): Plan {
    const id = `plan_${++this.counter}_${Date.now()}`;
    const tasks: PlanTask[] = taskTitles.map((title, index) => ({
      id: `${id}_task_${index}`,
      title,
      done: false,
      order: index,
      dependsOn: index > 0 ? [`${id}_task_${index - 1}`] : [],
    }));
    const plan: Plan = { id, goal, tasks, createdAt: new Date().toISOString() };
    this.plans.set(id, plan);
    return plan;
  }

  getPlan(planId: string): Plan | undefined {
    return this.plans.get(planId);
  }

  completeTask(planId: string, taskId: string): Plan | undefined {
    const plan = this.plans.get(planId);
    if (!plan) return undefined;
    const task = plan.tasks.find((t) => t.id === taskId);
    if (task) task.done = true;
    return plan;
  }

  nextTask(planId: string): PlanTask | undefined {
    const plan = this.plans.get(planId);
    if (!plan) return undefined;
    return plan.tasks
      .filter((t) => !t.done)
      .sort((a, b) => a.order - b.order)
      .find((t) =>
        (t.dependsOn ?? []).every(
          (depId) => plan.tasks.find((d) => d.id === depId)?.done
```

```
        )
      );
  }
}
```

src/lib/spark/memory/interface.ts

```
export type MemoryKind = "short_term" | "long_term" | "semantic" | "episodic" | "

export interface MemoryRecord {
  id: string;
  kind: MemoryKind;
  sessionId?: string;
  userId?: string;
  content: string;
  tags?: string[];
  createdAt: string;
  /**
   * Monotonically increasing insertion order. `createdAt` alone is not a
   * reliable sort key — multiple records can share the same millisecond
   * timestamp. `recall()` implementations must use `sequence` as the
   * tiebreaker so "newest first" holds even for same-millisecond writes.
   */
  sequence: number;
}

export interface MemoryQuery {
  kind?: MemoryKind;
  sessionId?: string;
  userId?: string;
  tag?: string;
  text?: string;
  limit?: number;
}

export interface MemoryService {
  remember(input: {
    kind: MemoryKind;
    content: string;
    sessionId?: string;
    userId?: string;
    tags?: string[];
  }): MemoryRecord;
```

```
  recall(query: MemoryQuery): MemoryRecord[];

  forget(id: string): boolean;

  clearSession(sessionId: string): number;

  size(): number;
}
```

src/lib/spark/memory/local.ts

```
import type { MemoryKind, MemoryQuery, MemoryRecord, MemoryService } from "./inte

/**
 * In-memory implementation covering short-term, long-term, semantic,
 * episodic and workspace memory kinds through a single flat store with
 * a `kind` discriminator. No persistence, no network calls.
 */
export class InMemoryMemoryService implements MemoryService {
  private records = new Map<string, MemoryRecord>();
  private counter = 0;

  remember(input: {
    kind: MemoryKind;
    content: string;
    sessionId?: string;
    userId?: string;
    tags?: string[];
  }): MemoryRecord {
    const sequence = ++this.counter;
    const record: MemoryRecord = {
      id: `mem_${sequence}_${Date.now()}`,
      kind: input.kind,
      sessionId: input.sessionId,
      userId: input.userId,
      content: input.content,
      tags: input.tags ?? [],
      createdAt: new Date().toISOString(),
      sequence,
    };
    this.records.set(record.id, record);
    return record;
  }
```

```
  recall(query: MemoryQuery): MemoryRecord[] {
    let results = Array.from(this.records.values());

    if (query.kind) results = results.filter((r) => r.kind === query.kind);
    if (query.sessionId)
      results = results.filter((r) => r.sessionId === query.sessionId);
    if (query.userId) results = results.filter((r) => r.userId === query.userId);
    if (query.tag) results = results.filter((r) => (r.tags ?? []).includes(query.
    if (query.text) {
      const needle = query.text.toLowerCase();
      results = results.filter((r) => r.content.toLowerCase().includes(needle));
    }

    results.sort((a, b) => {
      const timeDiff = new Date(b.createdAt).getTime() - new Date(a.createdAt).ge
      return timeDiff !== 0 ? timeDiff : b.sequence - a.sequence;
    });

    return query.limit ? results.slice(0, query.limit) : results;
  }

  forget(id: string): boolean {
    return this.records.delete(id);
  }

  clearSession(sessionId: string): number {
    let cleared = 0;
    for (const [id, record] of this.records) {
      if (record.sessionId === sessionId) {
        this.records.delete(id);
        cleared++;
      }
    }
    return cleared;
  }

  size(): number {
    return this.records.size;
  }
}
```

src/lib/spark/context/interface.ts

```
export interface EnvironmentContext {
```

```
  platform?: string;
  appVersion?: string;
  networkQuality?: "offline" | "poor" | "good" | "excellent";
}

export interface SessionContextSnapshot {
  sessionId: string;
  product: string;
  environment?: EnvironmentContext;
  attributes: Record<string, unknown>;
  updatedAt: string;
}

export interface ContextService {
  setEnvironment(sessionId: string, env: EnvironmentContext): void;
  setAttribute(sessionId: string, key: string, value: unknown): void;
  getSnapshot(sessionId: string, product: string): SessionContextSnapshot;
  clear(sessionId: string): boolean;
}
```

src/lib/spark/context/local.ts

```
import type {
  ContextService,
  EnvironmentContext,
  SessionContextSnapshot,
} from "./interface";

/**
 * Tracks per-session environmental/product awareness (in addition to
 * the user/session identity owned by the Identity service). Kept
 * separate from Identity because context changes far more often
 * (network quality, current screen, ephemeral attributes) than
 * identity does.
 */
export class LocalContextService implements ContextService {
  private environments = new Map<string, EnvironmentContext>();
  private attributes = new Map<string, Record<string, unknown>>();

  setEnvironment(sessionId: string, env: EnvironmentContext): void {
    this.environments.set(sessionId, env);
  }

  setAttribute(sessionId: string, key: string, value: unknown): void {
```

```
    const existing = this.attributes.get(sessionId) ?? {};
    existing[key] = value;
    this.attributes.set(sessionId, existing);
  }

  getSnapshot(sessionId: string, product: string): SessionContextSnapshot {
    return {
      sessionId,
      product,
      environment: this.environments.get(sessionId),
      attributes: this.attributes.get(sessionId) ?? {},
      updatedAt: new Date().toISOString(),
    };
  }

  clear(sessionId: string): boolean {
    const hadEnv = this.environments.delete(sessionId);
    const hadAttrs = this.attributes.delete(sessionId);
    return hadEnv || hadAttrs;
  }
}
```

src/lib/spark/knowledge/interface.ts

```
export interface KnowledgeDocument {
  id: string;
  title: string;
  content: string;
  tags?: string[];
  addedAt: string;
}

export interface KnowledgeQueryResult {
  document: KnowledgeDocument;
  score: number;
}

export interface KnowledgeService {
  addDocument(input: { title: string; content: string; tags?: string[] }): Knowle
  getDocument(id: string): KnowledgeDocument | undefined;
  removeDocument(id: string): boolean;
  /** Simple relevance query. Real embeddings can replace scoring later. */
  query(text: string, limit?: number): KnowledgeQueryResult[];
  size(): number;
```

```
}
```

src/lib/spark/knowledge/local.ts

```
import type {
  KnowledgeDocument,
  KnowledgeQueryResult,
  KnowledgeService,
} from "./interface";

/**
 * In-memory knowledge store with naive keyword-overlap scoring. This
 * stands in for a knowledge graph / embeddings-backed retrieval system;
 * the KnowledgeService interface is stable so a real backend can be
 * swapped in without changing consumers.
 */
export class LocalKnowledgeService implements KnowledgeService {
  private documents = new Map<string, KnowledgeDocument>();
  private counter = 0;

  addDocument(input: {
    title: string;
    content: string;
    tags?: string[];
  }): KnowledgeDocument {
    const doc: KnowledgeDocument = {
      id: `doc_${++this.counter}_${Date.now()}`,
      title: input.title,
      content: input.content,
      tags: input.tags ?? [],
      addedAt: new Date().toISOString(),
    };
    this.documents.set(doc.id, doc);
    return doc;
  }

  getDocument(id: string): KnowledgeDocument | undefined {
    return this.documents.get(id);
  }

  removeDocument(id: string): boolean {
    return this.documents.delete(id);
  }
```

```
  query(text: string, limit = 5): KnowledgeQueryResult[] {
    const queryWords = new Set(
      text.toLowerCase().split(/\W+/).filter(Boolean)
    );

    const scored: KnowledgeQueryResult[] = Array.from(
      this.documents.values()
    ).map((document) => {
      const docWords = `${document.title} ${document.content}`
        .toLowerCase()
        .split(/\W+/)
        .filter(Boolean);
      const overlap = docWords.filter((w) => queryWords.has(w)).length;
      const score = docWords.length ? overlap / docWords.length : 0;
      return { document, score };
    });

    return scored
      .filter((r) => r.score > 0)
      .sort((a, b) => b.score - a.score)
      .slice(0, limit);
  }

  size(): number {
    return this.documents.size;
  }
}
```

src/lib/spark/intelligence/search/interface.ts

```
export interface SearchResult {
  id: string;
  title: string;
  snippet: string;
  score: number;
}

export interface SearchService {
  search(query: string, limit?: number): Promise<SearchResult[]>;
}
```

src/lib/spark/intelligence/search/local.ts

```
import type { SearchService, SearchResult } from "./interface";
import type { KnowledgeService } from "../../knowledge/interface";

/**
 * Search is a thin, focused layer over Knowledge retrieval. It depends
 * on the KnowledgeService abstraction rather than reaching into Memory
 * directly, keeping the dependency graph shallow: Search -> Knowledge.
 */
export class LocalSearchService implements SearchService {
  constructor(private readonly knowledge: KnowledgeService) {}

  async search(query: string, limit = 5): Promise<SearchResult[]> {
    const matches = this.knowledge.query(query, limit);
    return matches.map((m) => ({
      id: m.document.id,
      title: m.document.title,
      snippet: m.document.content.slice(0, 160),
      score: m.score,
    }));
  }
}
```

src/lib/spark/intelligence/recommendations/interface.ts

```
export interface Recommendation {
  itemId: string;
  reason: string;
  score: number;
}

export interface RecommendationsService {
  recommend(input: {
    userId?: string;
    sessionId?: string;
    candidateItems: Array<{ id: string; tags?: string[] }>;
    basedOnTags?: string[];
    limit?: number;
  }): Recommendation[];
}
```

src/lib/spark/intelligence/recommendations/local.ts

```
import type { Recommendation, RecommendationsService } from "./interface";

/**
 * Deterministic tag-overlap recommender. No model call required; this
 * can be replaced with a learned ranker later behind the same interface.
 */
export class LocalRecommendationsService implements RecommendationsService {
  recommend(input: {
    userId?: string;
    sessionId?: string;
    candidateItems: Array<{ id: string; tags?: string[] }>;
    basedOnTags?: string[];
    limit?: number;
  }): Recommendation[] {
    const wanted = new Set(input.basedOnTags ?? []);

    const scored: Recommendation[] = input.candidateItems.map((item) => {
      const tags = item.tags ?? [];
      const overlap = tags.filter((t) => wanted.has(t)).length;
      const score = wanted.size ? overlap / wanted.size : 0;
      return {
        itemId: item.id,
        reason: overlap
          ? `Matches ${overlap} of ${wanted.size} preferred tag(s).`
          : "No tag overlap; ranked by default order.",
        score,
      };
    });

    return scored
      .sort((a, b) => b.score - a.score)
      .slice(0, input.limit ?? 10);
  }
}
```

src/lib/spark/intelligence/organization/interface.ts

```
export interface OrganizableItem {
  id: string;
  label: string;
  tags?: string[];
```

```
  createdAt?: string;
}

export interface OrganizedGroup {
  key: string;
  items: OrganizableItem[];
}

export interface OrganizationService {
  groupByTag(items: OrganizableItem[]): OrganizedGroup[];
  sortByRecency(items: OrganizableItem[]): OrganizableItem[];
}
```

src/lib/spark/intelligence/organization/local.ts

```
import type {
  OrganizableItem,
  OrganizedGroup,
  OrganizationService,
} from "./interface";

/** Deterministic local organization logic — grouping and sorting only. */
export class LocalOrganizationService implements OrganizationService {
  groupByTag(items: OrganizableItem[]): OrganizedGroup[] {
    const groups = new Map<string, OrganizableItem[]>();
    for (const item of items) {
      const tags = item.tags?.length ? item.tags : ["untagged"];
      for (const tag of tags) {
        if (!groups.has(tag)) groups.set(tag, []);
        groups.get(tag)!.push(item);
      }
    }
    return Array.from(groups.entries()).map(([key, groupItems]) => ({
      key,
      items: groupItems,
    }));
  }

  sortByRecency(items: OrganizableItem[]): OrganizableItem[] {
    return [...items].sort((a, b) => {
      const aTime = a.createdAt ? new Date(a.createdAt).getTime() : 0;
      const bTime = b.createdAt ? new Date(b.createdAt).getTime() : 0;
      return bTime - aTime;
    });
```

```
  }
}
```

src/lib/spark/intelligence/personalization/interface.ts

```
export interface UserPreference {
  key: string;
  value: unknown;
  updatedAt: string;
}

export interface PersonalizationService {
  setPreference(userId: string, key: string, value: unknown): UserPreference;
  getPreference(userId: string, key: string): UserPreference | undefined;
  getAllPreferences(userId: string): UserPreference[];
}
```

src/lib/spark/intelligence/personalization/local.ts

```
import type { PersonalizationService, UserPreference } from "./interface";

/** In-memory per-user preference store. */
export class LocalPersonalizationService implements PersonalizationService {
  private preferences = new Map<string, Map<string, UserPreference>>();

  setPreference(userId: string, key: string, value: unknown): UserPreference {
    if (!this.preferences.has(userId)) {
      this.preferences.set(userId, new Map());
    }
    const pref: UserPreference = { key, value, updatedAt: new Date().toISOString(
    this.preferences.get(userId)!.set(key, pref);
    return pref;
  }

  getPreference(userId: string, key: string): UserPreference | undefined {
    return this.preferences.get(userId)?.get(key);
  }

  getAllPreferences(userId: string): UserPreference[] {
    return Array.from(this.preferences.get(userId)?.values() ?? []);
  }
```

```
}
```

src/lib/spark/intelligence/writing/interface.ts

```
export type WritingTone = "neutral" | "formal" | "casual" | "concise";

export interface WritingRequest {
  prompt: string;
  tone?: WritingTone;
  maxLength?: number;
}

export interface WritingResult {
  text: string;
  tone: WritingTone;
  provider: string;
}

export interface WritingService {
  draft(request: WritingRequest): Promise<WritingResult>;
}
```

src/lib/spark/intelligence/writing/local.ts

```
import type { WritingRequest, WritingResult, WritingService } from "./interface";
import type { ProviderRegistry } from "../../providers/registry";

/** Delegates drafting to the resolved AI provider (mock by default). */
export class LocalWritingService implements WritingService {
  constructor(private readonly providers: ProviderRegistry) {}

  async draft(request: WritingRequest): Promise<WritingResult> {
    const provider = this.providers.resolve();
    const tone = request.tone ?? "neutral";

    const completion = await provider.complete({
      messages: [
        {
          role: "system",
          content: `You are Spark's writing module. Tone: ${tone}.`,
        },
```

```
        { role: "user", content: request.prompt },
      ],
      maxTokens: request.maxLength,
    });

    return { text: completion.text, tone, provider: completion.provider };
  }
}
```

src/lib/spark/intelligence/translation/interface.ts

```
export interface TranslationRequest {
  text: string;
  targetLocale: string;
  sourceLocale?: string;
}

export interface TranslationResult {
  text: string;
  targetLocale: string;
  sourceLocale?: string;
  provider: string;
}

export interface TranslationService {
  translate(request: TranslationRequest): Promise<TranslationResult>;
}
```

src/lib/spark/intelligence/translation/local.ts

```
import type {
  TranslationRequest,
  TranslationResult,
  TranslationService,
} from "./interface";
import type { ProviderRegistry } from "../../providers/registry";

/** Delegates translation to the resolved AI provider (mock by default). */
export class LocalTranslationService implements TranslationService {
  constructor(private readonly providers: ProviderRegistry) {}
```

```
  async translate(request: TranslationRequest): Promise<TranslationResult> {
    const provider = this.providers.resolve();

    const completion = await provider.complete({
      messages: [
        {
          role: "system",
          content: `You are Spark's translation module. Translate into ${request.
        },
        { role: "user", content: request.text },
      ],
    });

    return {
      text: completion.text,
      targetLocale: request.targetLocale,
      sourceLocale: request.sourceLocale,
      provider: completion.provider,
    };
  }
}
```

src/lib/spark/intelligence/summaries/interface.ts

```
export interface SummaryRequest {
  text: string;
  maxSentences?: number;
}

export interface SummaryResult {
  summary: string;
  originalLength: number;
  summaryLength: number;
}

export interface SummariesService {
  summarize(request: SummaryRequest): SummaryResult;
}
```

src/lib/spark/intelligence/summaries/local.ts

```
import type { SummaryRequest, SummaryResult, SummariesService } from "./interface

/**
 * Deterministic extractive summarizer (first-N-sentences heuristic).
 * No AI provider required. Can be upgraded to abstractive summarization
 * via a provider later without changing the interface.
 */
export class LocalSummariesService implements SummariesService {
  summarize(request: SummaryRequest): SummaryResult {
    const sentences = request.text
      .split(/(?<=[.!?])\s+/)
      .map((s) => s.trim())
      .filter(Boolean);

    const maxSentences = request.maxSentences ?? 3;
    const summary = sentences.slice(0, maxSentences).join(" ");

    return {
      summary,
      originalLength: request.text.length,
      summaryLength: summary.length,
    };
  }
}
```

src/lib/spark/trust/privacy/interface.ts

```
export interface PiiFinding {
  type: "email" | "phone" | "ssn" | "credit_card";
  match: string;
  index: number;
}

export interface PrivacyService {
  scanForPii(text: string): PiiFinding[];
  redact(text: string): string;
}
```

src/lib/spark/trust/privacy/local.ts

```
import type { PiiFinding, PrivacyService } from "./interface";
```

```
const PATTERNS: Array<{ type: PiiFinding["type"]; regex: RegExp }> = [
  { type: "email", regex: /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g },
  { type: "phone", regex: /\+?\d[\d\s().-]{7,}\d/g },
  { type: "ssn", regex: /\b\d{3}-\d{2}-\d{4}\b/g },
  { type: "credit_card", regex: /\b(?:\d[ -]*?){13,16}\b/g },
];

/** Local, regex-based PII detection and redaction. No network calls. */
export class LocalPrivacyService implements PrivacyService {
  scanForPii(text: string): PiiFinding[] {
    const findings: PiiFinding[] = [];
    for (const { type, regex } of PATTERNS) {
      let match: RegExpExecArray | null;
      const re = new RegExp(regex);
      while ((match = re.exec(text)) !== null) {
        findings.push({ type, match: match[0], index: match.index });
        if (!re.global) break;
      }
    }
    return findings;
  }

  redact(text: string): string {
    let redacted = text;
    for (const { regex } of PATTERNS) {
      redacted = redacted.replace(new RegExp(regex), "[REDACTED]");
    }
    return redacted;
  }
}
```

src/lib/spark/trust/security/interface.ts

```
export interface SecurityCheckResult {
  safe: boolean;
  reasons: string[];
}

export interface SecurityService {
  /** Basic heuristic scan for common injection/scam patterns. */
  checkInput(text: string): SecurityCheckResult;
}
```

src/lib/spark/trust/security/local.ts

```
import type { SecurityCheckResult, SecurityService } from "./interface";

const SUSPICIOUS_PATTERNS: Array<{ label: string; regex: RegExp }> = [
  { label: "possible script injection", regex: /<script[\s>]/i },
  { label: "possible SQL injection", regex: /(\bunion\b.*\bselect\b|;\s*drop\s+ta
  { label: "urgent wire-transfer scam language", regex: /wire (transfer|money) im
  { label: "credential phishing language", regex: /verify your (password|account)
];

/** Local heuristic security scanner. No network calls, no ML model. */
export class LocalSecurityService implements SecurityService {
  checkInput(text: string): SecurityCheckResult {
    const reasons = SUSPICIOUS_PATTERNS.filter((p) => p.regex.test(text)).map(
      (p) => p.label
    );
    return { safe: reasons.length === 0, reasons };
  }
}
```

src/lib/spark/automation/interface.ts

```
export interface AutomationTask {
  id: string;
  name: string;
  status: "pending" | "running" | "completed" | "failed";
  result?: unknown;
  error?: string;
}

export type AutomationHandler = (input?: unknown) => Promise<unknown>;

export interface AutomationService {
  registerTask(name: string, handler: AutomationHandler): void;
  run(name: string, input?: unknown): Promise<AutomationTask>;
  history(limit?: number): AutomationTask[];
}
```

src/lib/spark/automation/local.ts

```
import type {
  AutomationHandler,
  AutomationService,
  AutomationTask,
} from "./interface";

/**
 * In-process workflow/task runner. Registers named handlers and
 * executes them on demand, tracking status/result/error for diagnostics.
 * No external orchestration system is required at this stage.
 */
export class LocalAutomationService implements AutomationService {
  private handlers = new Map<string, AutomationHandler>();
  private tasks: AutomationTask[] = [];
  private counter = 0;

  registerTask(name: string, handler: AutomationHandler): void {
    this.handlers.set(name, handler);
  }

  async run(name: string, input?: unknown): Promise<AutomationTask> {
    const handler = this.handlers.get(name);
    const task: AutomationTask = {
      id: `task_${++this.counter}_${Date.now()}`,
      name,
      status: "pending",
    };
    this.tasks.push(task);

    if (!handler) {
      task.status = "failed";
      task.error = `No automation task registered under name "${name}".`;
      return task;
    }

    task.status = "running";
    try {
      task.result = await handler(input);
      task.status = "completed";
    } catch (err) {
      task.status = "failed";
      task.error = err instanceof Error ? err.message : String(err);
    }
    return task;
```

```
  }

  history(limit = 50): AutomationTask[] {
    return this.tasks.slice(-limit);
  }
}
```

src/lib/spark/learning/interface.ts

```
export interface FeedbackEntry {
  id: string;
  subject: string;
  rating: "positive" | "negative" | "neutral";
  note?: string;
  createdAt: string;
}

export interface LearningService {
  recordFeedback(input: {
    subject: string;
    rating: "positive" | "negative" | "neutral";
    note?: string;
  }): FeedbackEntry;

  getFeedback(subject: string): FeedbackEntry[];

  /** Simple aggregate score in [-1, 1] based on recorded feedback. */
  score(subject: string): number;
}
```

src/lib/spark/learning/local.ts

```
import type { FeedbackEntry, LearningService } from "./interface";

const RATING_WEIGHT: Record<FeedbackEntry["rating"], number> = {
  positive: 1,
  neutral: 0,
  negative: -1,
};

/**
```

```
 * In-memory feedback store with a simple aggregate scoring function.
 * This is the seed of continuous improvement / adaptive personalization —
 * it does not train any model, it just tracks signal for later use.
 */
export class LocalLearningService implements LearningService {
  private feedback: FeedbackEntry[] = [];
  private counter = 0;

  recordFeedback(input: {
    subject: string;
    rating: FeedbackEntry["rating"];
    note?: string;
  }): FeedbackEntry {
    const entry: FeedbackEntry = {
      id: `fb_${++this.counter}_${Date.now()}`,
      subject: input.subject,
      rating: input.rating,
      note: input.note,
      createdAt: new Date().toISOString(),
    };
    this.feedback.push(entry);
    return entry;
  }

  getFeedback(subject: string): FeedbackEntry[] {
    return this.feedback.filter((f) => f.subject === subject);
  }

  score(subject: string): number {
    const entries = this.getFeedback(subject);
    if (!entries.length) return 0;
    const total = entries.reduce((sum, e) => sum + RATING_WEIGHT[e.rating], 0);
    return total / entries.length;
  }
}
```

src/lib/spark/providers/types.ts

```
/**
 * AI Provider abstraction.
 *
 * Spark services never call a provider SDK directly — they only ever
 * talk to this interface. Concrete providers (OpenAI, Anthropic, Gemini,
 * local models, etc.) are adapters that implement it and are registered
```

```
 * through the ProviderRegistry.
 */

export interface AIProviderMessage {
  role: "system" | "user" | "assistant";
  content: string;
}

export interface AIProviderCompletionRequest {
  messages: AIProviderMessage[];
  maxTokens?: number;
  temperature?: number;
}

export interface AIProviderCompletionResult {
  text: string;
  provider: string;
  model?: string;
  usage?: {
    inputTokens?: number;
    outputTokens?: number;
  };
}

export interface AIProviderEmbeddingResult {
  vector: number[];
  provider: string;
  model?: string;
}

export interface AIProvider {
  /** Unique identifier, e.g. "mock", "openai", "anthropic" */
  readonly name: string;

  /** Whether this provider is currently usable (e.g. has credentials). */
  isAvailable(): boolean;

  complete(
    request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult>;

  embed?(text: string): Promise<AIProviderEmbeddingResult>;
}
```

src/lib/spark/providers/registry.ts

```
import type { AIProvider } from "./types";
import { MockProvider } from "./mock";

/**
 * Holds all registered AI providers and resolves which one to use.
 * Services depend on this registry (or on a single resolved AIProvider),
 * never on a concrete vendor class.
 */
export class ProviderRegistry {
  private providers = new Map<string, AIProvider>();
  private defaultName: string;

  constructor() {
    const mock = new MockProvider();
    this.providers.set(mock.name, mock);
    this.defaultName = mock.name;
  }

  register(provider: AIProvider, makeDefault = false): void {
    this.providers.set(provider.name, provider);
    if (makeDefault) {
      this.defaultName = provider.name;
    }
  }

  setDefault(name: string): void {
    if (!this.providers.has(name)) {
      throw new Error(`Cannot set default: provider "${name}" is not registered.`
    }
    this.defaultName = name;
  }

  get(name?: string): AIProvider {
    const key = name ?? this.defaultName;
    const provider = this.providers.get(key);
    if (!provider) {
      throw new Error(`Provider "${key}" is not registered.`);
    }
    return provider;
  }

  /** Returns the default provider if available, else falls back to mock. */
  resolve(): AIProvider {
    const preferred = this.providers.get(this.defaultName);
```

```
    if (preferred && preferred.isAvailable()) {
      return preferred;
    }
    return this.providers.get("mock")!;
  }

  list(): Array<{ name: string; available: boolean; isDefault: boolean }> {
    return Array.from(this.providers.values()).map((p) => ({
      name: p.name,
      available: p.isAvailable(),
      isDefault: p.name === this.defaultName,
    }));
  }
}
```

src/lib/spark/providers/mock.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
  AIProviderEmbeddingResult,
} from "./types";

/**
 * Deterministic, offline provider. This is the default provider so that
 * Spark builds and runs with zero external dependencies and zero API keys.
 * It never makes a network call.
 */
export class MockProvider implements AIProvider {
  readonly name = "mock";

  isAvailable(): boolean {
    return true;
  }

  async complete(
    request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    const lastUser = [...request.messages]
      .reverse()
      .find((m) => m.role === "user");

    return {
```

```
      text: `[mock:${this.name}] acknowledged ${
        lastUser ? lastUser.content.length : 0
      } chars of input.`,
      provider: this.name,
      model: "mock-1",
      usage: { inputTokens: 0, outputTokens: 0 },
    };
  }

  async embed(text: string): Promise<AIProviderEmbeddingResult> {
    // Deterministic pseudo-embedding derived from character codes, so
    // the same input always yields the same vector without any model.
    const dims = 16;
    const vector = new Array(dims).fill(0);
    for (let i = 0; i < text.length; i++) {
      vector[i % dims] += text.charCodeAt(i);
    }
    const max = Math.max(1, ...vector.map(Math.abs));
    return {
      vector: vector.map((v) => v / max),
      provider: this.name,
      model: "mock-embed-1",
    };
  }
}
```

src/lib/spark/providers/openai.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
} from "./types";

/**
 * Placeholder adapter for OpenAI. Contains no SDK dependency and makes
 * no network calls. Swap in a real implementation later without
 * changing any Spark service code — they only depend on AIProvider.
 */
export class OpenAIProvider implements AIProvider {
  readonly name = "openai";

  constructor(private readonly apiKey?: string) {}
```

```
  isAvailable(): boolean {
    return Boolean(this.apiKey);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "OpenAIProvider is a placeholder and has no API key configured."
      );
    }
    throw new Error("OpenAIProvider.complete() is not yet implemented.");
  }
}
```

src/lib/spark/providers/anthropic.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
} from "./types";

/**
 * Placeholder adapter for Anthropic. Contains no SDK dependency and
 * makes no network calls.
 */
export class AnthropicProvider implements AIProvider {
  readonly name = "anthropic";

  constructor(private readonly apiKey?: string) {}

  isAvailable(): boolean {
    return Boolean(this.apiKey);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "AnthropicProvider is a placeholder and has no API key configured."
      );
```

```
    }
    throw new Error("AnthropicProvider.complete() is not yet implemented.");
  }
}
```

src/lib/spark/providers/gemini.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
} from "./types";

/** Placeholder adapter for Google Gemini. No SDK dependency, no network calls. *
export class GeminiProvider implements AIProvider {
  readonly name = "gemini";

  constructor(private readonly apiKey?: string) {}

  isAvailable(): boolean {
    return Boolean(this.apiKey);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "GeminiProvider is a placeholder and has no API key configured."
      );
    }
    throw new Error("GeminiProvider.complete() is not yet implemented.");
  }
}
```

src/lib/spark/providers/local.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
```

```
} from "./types";

/**
 * Placeholder adapter for a local/self-hosted model (e.g. Ollama, a
 * local llama.cpp server). No network calls are made in this phase.
 */
export class LocalModelProvider implements AIProvider {
  readonly name = "local";

  constructor(private readonly endpoint?: string) {}

  isAvailable(): boolean {
    return Boolean(this.endpoint);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "LocalModelProvider is a placeholder and has no endpoint configured."
      );
    }
    throw new Error("LocalModelProvider.complete() is not yet implemented.");
  }
}
```